



TRAINING PROGRAMME



Lessons 4 & 5

Object Calisthenics

"Learn the rules like a pro, so you can break them like an artist."— Pablo Picasso

Object Calisthenics

The word ***calisthenics*** comes from the ancient Greek words
kalos (κάλλος), which means "beauty"
and
sthenos (σθένος), meaning "strength".



Object Calisthenics rules

- Only one level of indentation per method
- Don't use the ELSE keyword
- Wrap all primitives and strings
- First class collections (wrap all collections)
- Only one dot per line ~~dog.Body.Tail.Wag()~~ => `dog.ExpressHappiness()`
- No abbreviations
- Keep all entities small
[10 files per package, 50 lines per class, 5 lines per method, 2 arguments per method]
- No classes with more than two instance variables
- No public getters/setters/properties



Object Calisthenics: WHY?

Only one level of indentation per method

- helps keeping methods focused on one thing
- helps reducing the size of units

Don't use the ELSE keyword

- helps maintaining a single main execution lane
- promote using polymorphism for handling complex conditional cases

Wrap all primitives and strings

- Make the code more explicit by expressing intent through types that become behaviour attractors



Object Calisthenics: WHY?

First class collections (wrap all collections)

- any behaviour specific to the collection will be attracted to a single place (filtering, sorting, joins, etc.)
- changing the internal representation won't affect clients, improving decoupling

No getters/setters/properties

- aligns with the idea of OO Programming as a network of entities that collaborate by passing messages (calling methods) on each other
- promote encapsulation and a “tell, don't ask” approach



Object Calisthenics: WHY?

One dot per line

- exposes intent hiding implementation, a “tell, don’t ask” approach
- reduces the amount of knowledge you should have to enable a behaviour

Don’t abbreviate

- if we need a long name, it’s likely we are missing a concept: make it explicit
- abbreviations can be confusing, so it’s better to be clear
- typing is not the bottleneck

Keep entities small

- Small classes focus on doing just one thing, making them easier to understand and reuse



Object Calisthenics: WHY?

No classes with more than two instance variables

- the more instance variables, the lower the Cohesion within the class
- usually classes with more than one instance variable are orchestrators, and classes with one instance variables are actuators: separate the responsibilities

Heuristics

- ***Tell don't ask*** → Tell objects to perform actions instead of asking them for data and processing it elsewhere. Similar to no getters/setters.
- ***Law of Demeter*** → Each component should only talk to its friends; don't talk to strangers. Similar to one dot per line.



Object Calisthenics rules

- Only one level of indentation per method
- Don't use the ELSE keyword
- Wrap all primitives and strings
- First class collections (wrap all collections)
- Only one dot per line ~~dog.Body.Tail.Wag()~~ => `dog.ExpressHappiness()`
- No abbreviations
- Keep all entities small
[10 files per package, 50 lines per class, 5 lines per method, 2 arguments per method]
- No classes with more than two instance variables
- No public getters/setters/properties



Object Calisthenics



Tic Tac Toe kata

Rules:

- Players alternate placing X's and O's on a 3x3 board, with X starting first
- Players cannot play on already taken positions
- A player wins when it has three symbols in a row, either
 - Horizontally
 - Vertically
 - Diagonally
- If all squares are taken and there are not three symbols in a row, then the outcome is a draw

Caveat: try to make invalid state unrepresentable





Object Calisthenics

Game of Life kata

The universe of the Game of Life is an infinite two-dimensional grid. Any cell of the grid is in one of two possible states: dead or alive. Every cell interacts with its eight neighbours, which are the cells that are horizontally, vertically, or diagonally adjacent.

The universe evolves with time in steps.

At each step (or tick), the following transitions occur:

- Any live cell with fewer than two live neighbours dies, as if caused by under-population.

- Any live cell with two or three live neighbours should live on to the next generation.

- Any live cell with more than three live neighbours dies, as if by over-population.

- Any dead cell with exactly three live neighbours becomes a live cell, as if by reproduction.

The initial pattern constitutes the seed of the system. The first generation is created by applying the above rules simultaneously to every cell in the seed.

The rules continue to be applied repeatedly to create further generations.

